O-Laya: Fast, Ontology-Grounded System 1 Decisions with a Lean 4-Verified Axiom Projection Kernel and Inference-Free W3C PROV-O Workflow Replay

Dionisio Chiuratto Agourakis
dionisio@jquant.com.br
https://github.com/josaum/olaya · DOI: 10.5281/zenodo.23588930

October 2026 · Version 0.7.0 (`leio-decision`) · Zenodo Record 23588931

Abstract

Software pipelines and autonomous AI agents routinely execute high-frequency *System 1 decisions*: routing operational incidents, grounding messy product descriptions into standardized taxonomies (e.g., GS1 GPC, Schema.org, OBO Foundry), gating workflow branches, and aligning enterprise schemas. Existing approaches force an unsatisfactory trade-off between (i) autoregressive Large Language Models (LLMs), which incur 500 ms–3 s latency, non-deterministic sampling, out-of-vocabulary class hallucinations, and zero formal consistency guarantees, and (ii) flat vector retrieval or unconstrained neural classifiers, which routinely violate class-hierarchy monotonicity ($P(\text{child}) > P(\text{parent})$) and mutual exclusion (`owl:disjointWith`).

We present **O-Laya** (`leio-decision`), a neuro-symbolic System 1 decision engine that conditions inference directly on an OWL 2 RL / RDF / SKOS TBox at runtime. Given an unstructured or structured observation, O-Laya retrieves an active subgraph via hybrid lexical IDF and path-context bi-encoder Reciprocal Rank Fusion (`google/embeddinggemma-2`), scores candidate concepts in 8–180 ms on CPU, and projects the resulting distribution through a **formally verified axiom projection kernel** whose core monotonicity and disjointness invariants are machine-checked in **Lean 4** (`project_verified`, 16 theorems, zero `sorry`). Furthermore, O-Laya introduces a declarative **TBox-bound workflow compiler** that cryptographically binds multi-step decision state machines to the SHA-256 digest of the governing ontology, emitting W3C PROV-O JSON-LD provenance traces that can be deterministically replayed and audited offline without loading a neural model. Evaluated on the external **OAEI 2023 Conference (`ra1`)** benchmark across 21 ontology pairs, O-Laya achieves **0.73 M1 F1** using zero-dependency lexical scoring, **0.75 M1 F1** with zero-shot `google/embeddinggemma-2` path-context embeddings (outperforming `LogMap` and `Matcha` at 0.72), and **0.80 M1 F1** (0.76 M3 F1) with cascade LLM-judge verification. Across Python, Rust (`olaya-core` / WebAssembly), and zero-dependency TypeScript/Node.js runtimes, O-Laya guarantees a **0.0% runtime axiom violation rate**.

Keywords: Neuro-Symbolic AI · Ontology Grounding · OWL 2 RL Reasoning · Formal Verification · Lean 4 · Ontology Alignment · Provenance & Replay · System 1 Decision Making

---

1 Introduction

Production software systems and tool-using AI agents execute thousands of rapid, bounded classification and routing decisions before—or instead of—invoking expensive deliberative reasoning ("System 2"). Following the dual-process framing of cognitive architectures [1], these fast, automatic classifications constitute **System 1 decisions**:

- **Taxonomy and Catalog Grounding:** Mapping noisy merchant SKUs, clinical findings, or documents to specific leaf concepts in large hierarchies (`GS1`, `Schema.org`, `OBO`, or enterprise OWL ontologies) while preserving ancestor rollups.
- **Operational Triage and Funnel Routing:** Assigning alerts, logs, or customer utterances to incident trees or RevOps stages with explicit abstention margins.
- **Deterministic Agent Guardrails:** Controlling branch transitions in multi-step workflows where every option and terminal intent must conform to a formal domain schema.

In practice, engineering teams currently rely on two brittle paradigms:

1. **Autoregressive LLM Prompting:** Prompting a generative LLM for every classification incurs high latency (0.5–3.0 s), substantial token cost at scale, sensitivity to prompt perturbations, and closed-world violations (hallucinating CURIEs or class names absent from the governing ontology).
2. **Unconstrained Vector Search and Flat Classifiers:** Embedding class labels and computing cosine similarity or flat softmax probabilities ignores the algebraic structure of the ontology. Specifically, flat scorers frequently assign higher probability to a subclass than to its superclass ($P(\text{ChilledYogurt}) > P(\text{Perishable})$) or assign simultaneous high probability mass to mutually exclusive classes ($A \sqcap B \sqsubseteq \bot$).

O-Laya resolves this tension by treating an OWL/RDF/SKOS ontology not as static documentation, but as an executable **domain constraints kernel**. O-Laya separates *candidate scoring* (which may be lexical, bi-encoder embedding-based, or neural) from *formal axiom projection* and *deterministic workflow replay*.

1.1 Contributions

This work makes five primary contributions:

1. **Lean 4-Verified Axiom Projection Kernel (Section 3):** We specify a deterministic calibrate-and-project operator over finite ontology subgraphs that enforces subsumption monotonicity ($P(c) \leq P(p)$ for all $c \sqsubseteq p$) and disjointness exclusion ($\min(P(a), P(b)) = 0 \vee \max(P(a), P(b)) < \tau$ for all $a \sqcap b \sqsubseteq \bot$). We formalize the core lifting and child-closed disjointness operators in Lean 4 (`lean/OlayaAxioms/`) with 16 machine-checked theorems and zero `sorry` axioms.
2. **Fixpoint W3C OWL 2 RL Schema & Instance Closure (Section 2):** Prior to candidate routing and projection, O-Laya materializes the deductive closure of the input ontology—including transitive `rdfs:subClassOf`, `owl:equivalentClass`, `owl:disjointWith`, `owl:AllDisjointClasses`, `owl:disjointUnionOf`, `owl:intersectionOf` (`scm-int`), `owl:unionOf` (`scm-uni`), and `owl:propertyChainAxiom` (`prp-spo2`)—so implicit hierarchy and exclusion constraints are enforced identically to explicit assertions.
3. **Bring-Your-Own-Ontology (`OLaya.from_ontology`) & Five System 1 Primitives (Section 4):** A unified API supporting five typed primitives (`concept`, `subsumes`, `choice`, `score`, `noul`) over any `.ttl`, `.owl`, `.rdf`, `.nt`, `.jsonld`, or `.json` ontology, combining lexical IDF and `google/embeddinggemma-2` root-to-leaf path embeddings via Reciprocal Rank Fusion (RRF).
4. **TBox-Bound Workflows & Inference-Free W3C PROV-O Replay (Section 5):** A declarative workflow compiler (`olaya.workflow`) that type-checks decision graphs against the TBox, binds execution traces to SHA-256 digests of both the workflow and the ontology, and verifies historical traces offline (`olaya.workflow_replay`) without invoking any model or tokenizer.

5. **Tri-Runtime Parity & Reproducible Benchmarks (Sections 6–7):** Shared golden test vectors (`spec/axioms_golden.json`, `spec/decision_parity.json`) enforce mathematical parity across Python, Rust (`olaya-core` + `owl-fast-core` + `wasm32`), and zero-dependency TypeScript/Node.js, validated on OAEI 2023 Conference (`ra1`) and four domain routing benchmarks.

2 Formal Model and W3C OWL 2 RL Closure

Definition 2.1 (Ontology TBox)**.** *An ontology TBox is a tuple*

$$\mathcal{T} = (\mathcal{C}, \mathcal{R}, \sqsubseteq, \equiv, \perp, \mathrm{dom}, \mathrm{rng}, \mathcal{A}_{\mathrm{int}}, \mathcal{A}_{\mathrm{uni}}, \mathcal{A}_{\mathrm{chain}})$$

where $\mathcal{C}$ is a finite set of named classes (identified by compact URIs / CURIEs), $\mathcal{R}$ is a finite set of object properties, $\sqsubseteq \subseteq (\mathcal{C} \times \mathcal{C}) \cup (\mathcal{R} \times \mathcal{R})$ is the directed subsumption relation, $\equiv$ is equivalence, and $\perp \subseteq \mathcal{C} \times \mathcal{C}$ is the symmetric irreflexive disjointness relation.

2.1 Deductive Closure Materialization

Before serving queries or compiling workflows, O-Laya's forward-chaining reasoner (`OWL2RLReasoner` in Python; `owl-fast-core` in Rust) computes the least fixpoint $\mathrm{Cl}(\mathcal{T})$ under the W3C OWL 2 RL rule set [2]:

- **Class Hierarchy and Equivalence (`scm-cls`, `scm-sco`, `scm-eqc1`, `scm-eqc2`):** Reflexive-transitive closure of $\sqsubseteq$ over $\mathcal{C}$, mutual subsumption for $C_1 \equiv C_2$, and cycle collapse into equivalence classes.
- **Intersection and Union Decompositions (`scm-int`, `scm-uni`, `owl:disjointUnionOf`):**
  - If $C \sqsubseteq C_1 \sqcap \cdots \sqcap C_n$ (or $C \equiv C_1 \sqcap \cdots \sqcap C_n$), then $C \sqsubseteq C_i$ for all $1 \le i \le n$.
  - If $C \equiv C_1 \sqcup \cdots \sqcup C_n$, then $C_i \sqsubseteq C$ for all $1 \le i \le n$.
  - If C is an `owl:disjointUnionOf` $(C_1, \ldots, C_n)$, then $C_i \sqsubseteq C$ for all i and $(C_i, C_j) \in \perp$ for all $i \neq j$.
- **Disjointness Downward Propagation (`scm-dco`, `owl:AllDisjointClasses`):** If $(A, B) \in \perp$ and $A' \sqsubseteq A$, $B' \sqsubseteq B$, then $(A', B') \in \perp$. Any class C with $(C, C) \in \perp$ or $C \sqsubseteq$ `owl:Nothing` is recorded in `unsatisfiable_classes`, flagging $\mathrm{Cl}(\mathcal{T})$ as deductively inconsistent when non-empty.
- **Property Hierarchy, Domains, Ranges, and Chains (`scm-spo`, `scm-dom1/2`, `scm-rng1/2`, `prp-spo2`, `prp-fp`, `prp-ifp`, `prp-pdw`, `prp-asyp`, `prp-irp`):** Propagates domain and range class constraints across super-properties and evaluates multi-hop property chains $P_1 \circ \cdots \circ P_k \sqsubseteq P$ over ABox assertions.

Furthermore, $\mathcal{T}$ is assigned a format- and order-independent cryptographic digest:

$$\mathrm{fp}(\mathcal{T}) = \text{SHA-256}(\text{CanonicalJSON}(\mathcal{T})), \tag{1}$$

which cryptographically binds checkpoints, domain packs, and workflow execution traces to the exact TBox topology.

3 The Verified Axiom Projection Kernel

Given an observation x and a routed active subgraph $\mathcal{G}_K = (V_K, E_{\sqsubseteq}, E_{\perp})$ of $K = |V_K|$ candidate concepts with raw model or similarity logits $\mathbf{l} = (l_0, \ldots, l_{K-1}) \in \mathbb{R}^K$, O-Laya transforms $\mathbf{l}$ into an axiom-compliant assignment $\mathbf{q} \in [0, 1]^K$.

3.1 Five-Step Calibrate-and-Project Operator

The normative specification (`spec/AXIOMS.md`) defines five deterministic steps:

Step 1: Cardinality-Aware (or Fitted) Temperature Scaling. Without a fitted checkpoint temperature, the heuristic temperature is determined by candidate cardinality K:

$$T_{\text{card}}(K) = \begin{cases} 0.85 & \text{if } K \leq 2, \\ 0.95 & \text{if } 2 < K \leq 5, \\ 1.10 & \text{if } 5 < K \leq 10, \\ 1.25 & \text{if } K > 10. \end{cases} \tag{2}$$

When a checkpoint supplies an empirically calibrated temperature $T_{\text{fit}} > 0$ (fit via negative log-likelihood minimization on a held-out calibration split), logits are pre-scaled by $T_{\text{card}}(K)/T_{\text{fit}}$ so the effective softmax temperature is exactly T_{fit}.

Step 2: Temperature Softmax.

$$p_i = \frac{\exp((l_i - \max_j l_j)/T)}{\sum_{j=0}^{K-1} \exp((l_j - \max_m l_m)/T)}, \qquad i \in \{0, \ldots, K-1\}. \tag{3}$$

Step 3: Subsumption Monotonicity Lifting (`lift`). We compute the least monotone function $\mathbf{u} \geq \mathbf{p}$ over the poset $(V_K, E_{\sqsubseteq})$ by iterating until convergence:

$$\forall (c, p) \in E_{\sqsubseteq} : \qquad u_p \leftarrow \max(u_p, u_c). \tag{4}$$

For `concept` questions in the Python engine (`project_hierarchical_distribution`), only the specific leaf concepts $L_K \subseteq V_K$ (or routed concepts under `candidate_policy="routed"`) enter the softmax distribution $\mathbf{p}$, and each ancestor's marginal probability is derived by downward mass aggregation:

$$u_a = \sum_{c \in \text{desc}^*(a) \cap L_K} p_c, \tag{5}$$

ensuring $u_p \geq u_c$ holds by construction while $\sum_{c \in L_K} p_c = 1$.

Step 4: Child-Closed Disjointness Exclusion (`disjointAll`). With exclusion threshold $\tau = 0.6$, for each ordered disjoint pair $(a, b) \in E_{\perp}$ $(a < b)$:

- If $\max(u_a, u_b) \geq \tau$, let the losing concept be $w = b$ if $u_a \geq u_b$ (ties broken deterministically by slot index) and $w = a$ if $u_b > u_a$.
- Set $q_v \leftarrow 0$ for every $v \in \text{desc}^*(w) = \{v \in V_K \mid v \sqsubseteq^* w\}$.

Because zeroing is *child-closed* (every descendant of w is zeroed simultaneously), zeroing never creates a child c with $q_c > q_p$, preserving the subsumption monotonicity established in Step 3.

Step 5: Pre-Lift, Post-Disjointness Normalized-Entropy Confidence. Measuring Shannon entropy directly on the lifted scores $\mathbf{u}$ would artificially penalize deep hierarchies (since a leaf's mass is copied to all its ancestors, making a high-certainty leaf appear uniform across its lineage). Instead, O-Laya masks the pre-lift distribution $\mathbf{p}$ by the surviving support of $\mathbf{q}$:

$$\tilde{p}_i = \begin{cases} p_i & \text{if } q_i > 0, \\ 0 & \text{otherwise,} \end{cases} \qquad \hat{p}_i = \frac{\tilde{p}_i}{\sum_j \tilde{p}_j}, \tag{6}$$

$$\text{confidence}(\mathbf{p}, \mathbf{q}) = \text{clip}_{[0,1]}\left(1 - \frac{-\sum_{i:\, \hat{p}_i > 0} \hat{p}_i \ln \hat{p}_i}{\ln K}\right). \tag{7}$$

3.2 Machine-Checked Formal Proof in Lean 4

Steps 3 and 4 and the runtime verifier `verify_axioms` are formalized over arbitrary non-negative integer scores in core Lean 4 (`lean/OlayaAxioms/Basic.lean`, zero external dependencies, zero `sorry`). Table 1 summarizes the verified theorems.

Table 1: Core machine-checked theorems in `lean/OlayaAxioms/Basic.lean` (Lean 4, zero `sorry`).

Theorem Identifier	**Formal Property Established**
`lift_monotone`	For any edge list `edges` with sufficient fuel, `lift edges fuel s` satisfies $s'(c) \leq s'(p)$ for all $(c, p) \in$ `edges`.
`le_lift` & `lift_least`	`lift` dominates the input ($s \leq \text{lift}(s)$) and is the *least* monotone upper bound: for any monotone $m \geq s$, $\text{lift}(s) \leq m$.
`lift_idempotent_of_monotone`	Any already-monotone assignment is a fixed point of `lift`.
`zeroSubtree_preserves_monotone`	Zeroing the downward closure $\text{desc}^*(w)$ of any node w preserves monotonicity across all edges $(c, p) \in E_{\sqsubseteq}$.
`disjointAll_preserves_monotone`	Sequential application of child-closed disjointness exclusion over all $(a, b) \in E_{\perp}$ preserves subsumption monotonicity.
`disjointAll_resolves_pairs`	After `disjointAll`, every pair $(a, b) \in E_{\perp}$ satisfies $\min(q_a, q_b) = 0 \vee \max(q_a, q_b) < \tau$.
`project_verified`	**End-to-End Soundness:** For any non-negative score vector $\mathbf{s}$, $\text{verifyAxioms}(\text{project}(\mathbf{s})) =$ `true`.

Theorem 3.1 (Soundness of Axiom Projection — `project_verified`)**.** *Let $E_{\sqsubseteq}$ and $E_{\perp}$ be finite lists of subsumption and disjointness edges over K concept slots, and let $\mathbf{s} : \mathbb{N} \to \mathbb{N}$ be any non-negative score assignment. When* lift *reaches its fixed point $\mathbf{u} = \text{lift}(E_{\sqsubseteq}, \mathbf{s})$, the projected assignment $\mathbf{q} = \text{disjointAll}(E_{\sqsubseteq}, E_{\perp}, \tau, \mathbf{u})$ satisfies both:*

(i) $\forall (c, p) \in E_{\sqsubseteq},\ q_c \leq q_p$ *(subsumption monotonicity), and*

(ii) $\forall (a, b) \in E_{\perp},\ \min(q_a, q_b) = 0 \ \vee\ \max(q_a, q_b) < \tau$ *(disjointness exclusion).*

In continuous integration, `lean4checker` audits the compiled environment to verify that `project_verified` depends only on Lean's standard foundational axioms (`propext`, `Quot.sound`), and `tests/test_lean_oracle.py` cross-checks the Python floating-point kernel against the compiled Lean 4 executable oracle.

4 System Architecture and The Five System 1 Primitives

4.1 Hybrid Candidate Routing and Path-Context Bi-Encoder Scoring

Real-world ontologies contain hundreds to tens of thousands of classes (`Schema.org` $\approx$ 800, `GS1` $\approx$ 4,000, `MONDO/ChEBI` $>$ 20,000). Scoring all classes with a cross-encoder on every call is prohibitive; scoring bare class labels ignores taxonomic context (e.g., a class labeled `"Standard"` under `ShippingMethod` versus `RoomCategory`).

When instantiated via `OLaya.from_ontology(path)`, O-Laya constructs a **path-context representation** for every class $c \in \mathcal{C}$:

$$\text{ctx}(c) = \text{label}(a_0) \ \texttt{>}\ \text{label}(a_1) \ \texttt{>}\ \ldots \ \texttt{>}\ \text{label}(c)\ [\text{synonyms}(c)]\ \texttt{:}\quad \text{description}(c), \tag{8}$$

where $(a_0, a_1, \ldots, c)$ is the canonical root-to-leaf lineage in $\mathcal{T}$. At initialization, all $\text{ctx}(c)$ strings are embedded once using `google/embeddinggemma-2` (or a deterministic hash/lexical fallback in air-gapped environments). At query time:

1. **Lexical Inverted Index:** Computes smoothed IDF token overlap over labels (weight 3.0), synonyms (2.5), and descriptions (1.5).
2. **Dense Bi-Encoder Retrieval:** Computes cosine similarity between the query embedding and pre-computed path-context embeddings.
3. **Reciprocal Rank Fusion (RRF) and Ancestor Closure:** Merges lexical and dense rankings via RRF ($k_{\text{rrf}} = 60$) to select the top-K candidates and adds their ancestor closure up to the requested `root` CURIE to form the active subgraph $\mathcal{G}_K$.

4.2 The Five Unified `system_one` Primitives

A single `engine.system_one(state, questions)` call evaluates arbitrary combinations of five typed decision primitives (Table 2).

Table 2: The five typed System 1 decision primitives exposed by `engine.system_one`.

Primitive	TBox Binding	Mathematical Semantics & Output Contract
`concept`	`root` $\in \mathcal{C}$	Grounds `state` to the most specific leaf `curie` under `root`, returning root-to-leaf `lineage`, leaf distribution `candidate_probabilities`, ancestor marginals `probabilities`, `confidence`, and `axioms_verified`.
`subsumes`	`concept` $C \in \mathcal{C}$	Bernoulli probability $P(x \in \text{desc}^*(C))$ that the observation is an instance of class C or any of its descendants.
`choice`	`options` $\subseteq \text{desc}^*(r)$	Selects one mutually exclusive option from a discrete set with normalized probabilities and tie/margin abstention.
`score`	`criteria` $(c_0, \ldots, c_{M-1})$	Expected continuous level $\mathbb{E}[Y] = \sum_{m=0}^{M-1} m \cdot P(Y = m)$ across an ordered rubric of M classes or criteria levels.
`noul`	`concept` / `root`	Binary proposition probability $P(\text{true}) \in [0, 1]$ with explicit negation and out-of-domain abstention semantics.

5 Declarative TBox-Bound Workflows and Inference-Free Replay

Prompt chains and imperative agent routers intertwine domain schemas, model inference, and branching logic, making post-hoc compliance auditing impossible without re-running (often non-deterministic) models. O-Laya's workflow subsystem (`olaya.workflow` and `olaya.workflow_replay`) enforces a strict three-phase separation between **static TBox compilation**, **bounded live execution**, and **inference-free replay**:

1. **Static TBox Type-Checking (`compile_workflow`):** A workflow document is a bounded directed acyclic graph (1–128 nodes; cycles and unreachable nodes are rejected at compile time). Every decision node's bound `root`, `concept`, `options`, `criteria`, categorical route keys, and terminal `emit` resource types and properties are validated against $\text{Cl}(\mathcal{T})$:
   - Option and route classes must be subclasses of the node's declared `root` ($c \in \text{desc}^*(r)$).
   - Emitted properties must satisfy `rdfs:domain` and `rdfs:range` subsumption constraints over $\mathcal{T}$.
   - Numeric routes (`subsumes`, `score`, `noul`) are checked for non-empty, non-overlapping interval partitions.
   - The compiler outputs an immutable `CompiledWorkflow` containing the canonical workflow SHA-256 (`workflow_hash`) and TBox SHA-256 (`tbox_hash`).

2. **Safe Execution and Abstention Gating (`run_workflow`):** At each step, the interpreter verifies that the engine's live TBox digest matches `tbox_hash` (preventing mid-flight schema drift), invokes `system_one`, and applies acceptance gates (`min_confidence`, `min_margin`, `require_axioms`). If a decision violates an acceptance threshold, ties at the argmax, or encounters a backend error, execution deterministically transitions to the node's declared `on_abstain` or `on_error` branch. Terminal nodes emit typed `wf:ActionIntent` records rather than executing side effects.

3. **W3C PROV-O JSON-LD Emission and Offline Replay (`replay_workflow`):** Every execution emits a self-contained JSON-LD 1.1 `@graph` typed with W3C `prov:Activity`, `prov:Entity`, `wf:WorkflowRun`, `wf:DecisionActivity`, `wf:DecisionRecord`, and `wf:ActionIntent`—recording the decision distributions and statuses *without* persisting sensitive raw input state. Given only (`compiled_workflow`, `trace_jsonld`), `replay_workflow` constructs a zero-model `_ReplayEngine`, re-runs the compiled transition policy, and mathematically verifies that the recorded distributions, acceptance gates, and branch transitions reproduce the exact trace and terminal intent.

6 Empirical Evaluation

6.1 External Benchmark: OAEI 2023 Conference Track (`ra1`)

We evaluate O-Laya on the standard **Ontology Alignment Evaluation Initiative (OAEI) Conference Track** [4] using reference alignment `ra1` across all **21 ontology pairs** (7 OntoFarm ontologies: `cmt`, `conference`, `confOf`, `edas`, `ekaw`, `iasted`, `sigkdd`; **259 class correspondences** and **46 property correspondences**, 305 total). Following the official OAEI protocol, mutual-best 1:1 correspondences above a threshold fixed *a priori* ($\tau = 0.50$) or selected via Leave-One-Pair-Out Cross-Validation (LOPO-CV) are micro-averaged across all 21 pairs for classes (**M1**), properties (**M2**), and combined (**M3**).

Table 3: Micro-averaged Precision, Recall, and F1 on the OAEI 2023 Conference benchmark (`ra1`, 21 ontology pairs, 259 class + 46 property correspondences). O-Laya configurations are shown in bold.

System / Configuration	M1 P	M1 R	M1 F1 (Classes)	M3 P	M3 R	M3 F1 (All)
GraphMatcher (OAEI 2023 SOTA)	0.78	0.89	**0.83**	0.76	0.80	**0.78**
O-Laya Hybrid + LLM Judge (`qwen3.8-max`, LOPO-CV)	**0.79**	**0.82**	**0.80**	**0.73**	**0.79**	**0.76**
SORBETMtch (OAEI 2023)	0.78	0.75	0.76	0.78	0.64	0.70
O-Laya Hybrid (`embeddinggemma-2`, path text, $\tau = 0.50$)	**0.82**	**0.69**	**0.75**	**0.72**	**0.66**	**0.69**
O-Laya Hybrid (`intfloat/e5-small-v2`, LOPO-CV)	**0.81**	**0.69**	**0.75**	**0.69**	**0.66**	**0.68**
O-Laya Hybrid + Structure (`embeddinggemma-2`, LOPO-CV)	**0.80**	**0.71**	**0.75**	**0.69**	**0.67**	**0.68**
O-Laya Lexical ($\tau = 0.50$, zero ML dependencies)	**0.83**	**0.64**	**0.73**	**0.73**	**0.60**	**0.66**
LogMap (OAEI 2023)	0.84	0.63	0.72	0.81	0.58	0.68
Matcha (OAEI 2023)	0.76	0.68	0.72	0.70	0.64	0.67
edna (OAEI baseline)	0.88	0.54	0.67	0.79	0.47	0.59
StringEquiv (OAEI baseline)	0.88	0.50	0.64	0.80	0.43	0.56

Table 3 highlights three results:

1. Even with **zero ML dependencies**, `olaya-lexical` achieves **0.73 M1 F1**, outperforming `StringEquiv` (0.64) and `edna` (0.67) and edging past `LogMap` and `Matcha` (0.72) on class alignment via token normalization, CamelCase/snake_case decompounding, and inverted-index IDF weighting.

2. Conditioning `google/embeddinggemma-2` on root-to-leaf **path-context strings** (`olaya-hybrid`) lifts M1 F1 to **0.75** ($P = 0.82, R = 0.69$) at the *a priori* $\tau = 0.50$ threshold without task-specific training.

3. Adding a second-stage LLM verification pass over ambiguous candidate pairs (`olaya-hybrid + LLM judge`) reaches **0.80 M1 F1** and **0.76 M3 F1**, ranking second only to the supervised GNN `GraphMatcher` (0.83).

6.2 Full-Ontology Concept Grounding and Unseen-Class Generalization

To evaluate System 1 concept grounding across diverse hierarchies, we benchmark `OLaya.from_ontology` on four ontologies under both **batch-disjoint** splits (test observations drawn from held-out generation batches) and strict **class-disjoint** splits (where **0% of test target classes** appear in training, testing true zero-shot compositional generalization over the TBox hierarchy).

Table 4: Full-ontology Top-1 accuracy and runtime axiom violation rate across batch-disjoint and unseen class-disjoint splits.

Domain Ontology	Split Regime	Lexical IDF	Zero-Shot `bge-small`	LoRA `EmbeddingGemma 2`	Axiom Violations
Incident Triage	Batch-Disjoint	0.562	0.562	**0.812**	**0.0%**
Incident Triage	Class-Disjoint (Unseen)	0.562	0.562	**0.826**	**0.0%**
GS1 GPC Taxonomy	Per-Domain / Batch	0.281	0.375	**0.859**	**0.0%**
GS1 GPC Taxonomy	Class-Disjoint (Unseen)	0.281	0.375	**0.656**	**0.0%**
Schema.org	Joint / Batch	0.100	0.308	**0.750**	**0.0%**
Schema.org	Class-Disjoint (Unseen)	0.100	0.308	**0.606**	**0.0%**
RevOps Bowtie (`pt-BR`)	Cross-Lingual Zero-Shot	0.600	—	**1.000** (10/10)	**0.0%**

As shown in Table 4, LoRA fine-tuning of `google/embeddinggemma-2` (rank = 16, merged into base weights in 26 minutes on a single Apple Silicon GPU) improves full-ontology Top-1 accuracy by **+25.0 to +65.0 percentage points** over lexical routing while transferring **0.606–0.826 Top-1 accuracy** to completely unseen ontology classes via their root-to-leaf path embeddings.

6.3 Runtime Latency and Cross-Language Parity

Table 5 reports CPU latency on a standard 4-vCPU container without GPU acceleration (`benchmarks/results/router-latency.md`) alongside golden-vector parity across all four kernel targets.

Table 5: CPU latency profile (4-vCPU container) and cross-runtime golden vector parity (`spec/axioms_golden.json`).

Runtime & Scoring Backend	p50 Latency	p95 Latency	Memory	Golden Parity ($\epsilon = 10^{-6}$)
Rust `olaya-core` / WASM / Node (Lexical IDF)	**1–4 ms**	**6 ms**	< 15 MB	100%
Python `olaya` (Lexical IDF, zero-Torch)	**8–12 ms**	**16 ms**	< 40 MB	100%
Python `olaya` (`BAAI/bge-small-en-v1.5`, CPU)	**28 ms**	**36 ms**	≈ 180 MB	100%
Python `olaya` (`google/embeddinggemma-2`, CPU)	**115 ms**	**180 ms**	≈ 1.2 GB	100%

7 Scope Boundaries, Limitations, and Future Work

To maintain scientific and engineering transparency, we explicitly state what O-Laya guarantees and what remains outside its current formal scope:

1. **Boundary of the Lean 4 Formalization:** The Lean 4 proof core (`lean/OlayaAxioms/`) proves the exact correctness, monotonicity, least upper-bound optimality, and disjointness resolution of Steps 3 (`lift`) and 4 (`disjointAll`) and the verifier (`verifyAxioms`) over non-negative integer scores. Floating-point softmax (Step 2), Shannon entropy confidence (Step 5), and neural encoder representations lie outside the theorem prover's scope and

are validated empirically via property-based tests (`hypothesis`) and cross-runtime golden vectors.

2. **Empirical Calibration:** Normalized-entropy confidence is an information-theoretic concentration measure over the post-disjointness simplex. It becomes an empirically calibrated probability only when paired with a held-out temperature fit (`olaya.calibration`).

3. **Marker-Pooling Neural Head vs. Path-Context Bi-Encoder:** While O-Laya includes a trainable marker-pooling sequence head (`olaya.model.OntologyDecisionModel`), our ablation studies (`benchmarks/results/decision-head-summary.md`) show that path-context bi-encoder scoring (`OLaya.from_ontology`) substantially outperforms from-scratch marker heads in data-scarce regimes. Both positive and negative ablation results are archived in the repository.

8 Reproducibility and Software Availability

O-Laya (`leio-decision v0.7.0`) is released under the MIT License at https://github.com/josaum/olaya, archived on Zenodo under concept DOI 10.5281/zenodo.23588930 (version 0.7.0 record: 10.5281/zenodo.23588931), and published to PyPI and npm as `leio-decision`. All benchmarks, formal proofs, and cross-runtime parity checks can be reproduced directly from the repository:

```
# 1. Run Python test suite (460+ tests: OWL 2 RL, axiom golden vectors, workflows, replay, PII)
.venv/bin/python -m pytest -q

# 2. Verify the Lean 4 proof core (zero sorry, standard axioms via lean4checker)
cd lean && lake build && lake exe olaya-oracle

# 3. Run Rust workspace tests (olaya-core, owl-fast-core, FCA, WASM kernel)
cargo test --workspace

# 4. Run zero-dependency TypeScript / Node.js test suite
cd packages/olaya-node && bun test

# 5. Reproduce OAEI 2023 Conference (ra1) benchmark
.venv/bin/python scripts/benchmark_oaei.py
```

References

[1] D. Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[2] B. Motik, B. Cuenca Grau, I. Horrocks, Z. Wu, A. Fokoue, and C. Lutz. OWL 2 Web Ontology Language Profiles (Second Edition). W3C Recommendation, December 2012. https://www.w3.org/TR/owl2-profiles/.

[3] L. Moreau and P. Missier. PROV-DM: The PROV Data Model. W3C Recommendation, April 2013. https://www.w3.org/TR/prov-dm/.

[4] M. A. N. Pour, A. Algergawy, P. Buche, L. J. Castro, J. M. Chen, H. Coulibaly, et al. Results of the Ontology Alignment Evaluation Initiative 2023. In *Proceedings of the 18th International Workshop on Ontology Matching (OM 2023)*, CEUR Workshop Proceedings, Vol. 3591, pp. 97–139, 2023.

[5] E. Jiménez-Ruiz and B. Cuenca Grau. LogMap: Logic-Based and Scalable Ontology Matching. In *Proceedings of the 10th International Semantic Web Conference (ISWC 2011)*, pp. 273–288. Springer, 2011.

[6] D. Faria, M. C. Silva, P. Cotovio, L. Ferraz, L. Balbi, and C. Pesquita. Results for Matcha and Matcha-DL in OAEI 2023. In *Proceedings of the 18th International Workshop on Ontology Matching (OM 2023)*, CEUR Workshop Proceedings, Vol. 3591, pp. 164–170, 2023.

[7] L. de Moura and S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science, Vol. 12699, pp. 625–635. Springer, 2021.